

Оконные функции. Продвинутые методы работы с SQL



Оконные функции

PostgreSQL предлагает еще один способ вычислений, затрагивающих несколько записей, - **оконные функции**.

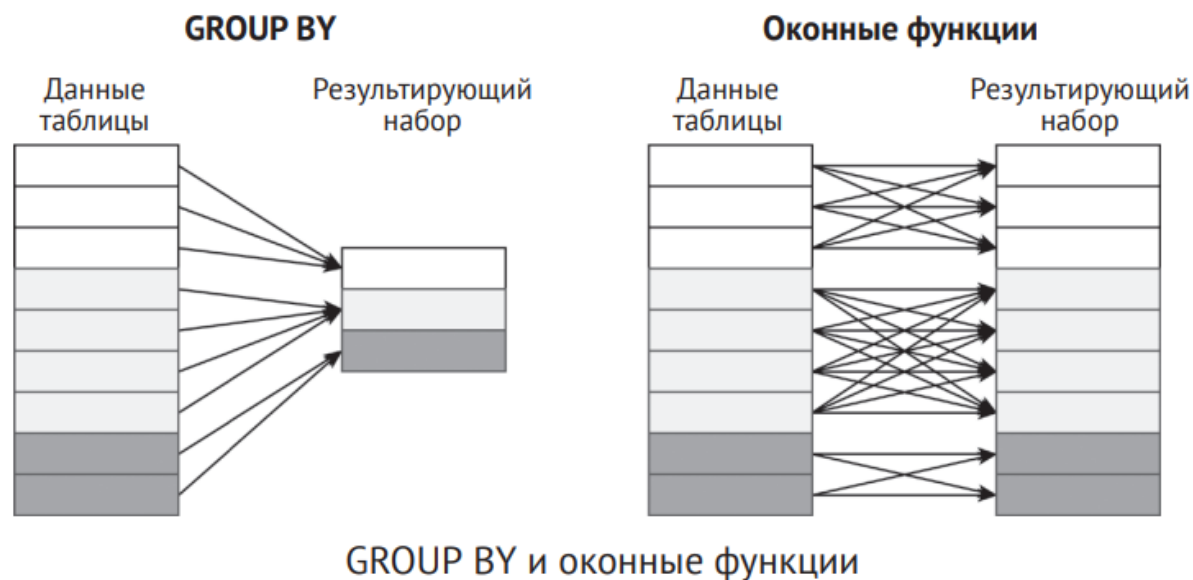
Эти функции предоставляют возможность производить вычисления на множестве строк, логически связанных с текущей строкой, т. е. имеющих то или иное отношение к ней.

- При работе с оконными функциями используются концепции **раздела (partition)** и **оконного кадра (window frame)**.
- Раздел включает в себя все строки выборки, имеющие в некотором смысле одинаковые свойства, например, одинаковые значения определенных выражений, задаваемых с помощью предложения PARTITION BY.
- Оконный кадр состоит из подмножества строк данного раздела и привязан к текущей строке.
- Для определения границ кадра важным является наличие предложения ORDER BY при формировании раздела.



Оконные функции

В случае группировки и агрегирования выводится одна запись для каждой группы входных записей. Оконные функции выполняют похожие действия, но для каждой записи. Количество записей на входе и на выходе одинаково.



На рисунке прямоугольниками представлены записи таблицы. Цвет прямоугольника обозначает значение поля, по которому производится группировка записей.

При использовании **GROUP BY** для каждого нового значения создается группа, которой соответствует одна запись в результате запроса.

Оконные функции дают доступ ко всем записям группы (которая в этом случае называется **разделом**), а количество записей на выходе такое же, как на входе.

При использовании оконных функций группировка не обязательна, хотя и возможна.



Оконные функции

Вызов оконной функции **всегда содержит предложение OVER**, следующее за названием и аргументами оконной функции. Это синтаксически отличает её от обычной или агрегатной функции.

Предложение OVER определяет, как именно нужно разделить строки запроса для обработки оконной функцией.

Предложение PARTITION BY, дополняющее OVER, указывает, что строки нужно разделить по группам или разделам, объединяя одинаковые значения выражений PARTITION BY.

Оконная функция вычисляется по строкам, попадающим в один раздел с текущей строкой.

Можно также определять порядок, в котором строки будут обрабатываться оконными функциями, используя ORDER BY в OVER.



Оконные функции

Строки, обрабатываемые оконной функцией, представляют собой «виртуальные таблицы», созданные из предложения FROM и затем прошедшие через фильтрацию и группировку WHERE и GROUP BY и, возможно, условие HAVING.

Например, строка, отфильтрованная из-за нарушения условия WHERE, не будет видна для оконных функций.

Запрос может содержать несколько оконных функций, разделяющих данные по-разному с помощью разных предложений OVER, но все они будут обрабатывать один и тот же набор строк этой виртуальной таблицы.

ORDER BY можно опустить, если порядок строк не важен.

Также возможно опустить PARTITION BY, в этом случае будет только один раздел, содержащий все строки.



Синтаксис

Оконные функции вычисляются после группировки и агрегирования. Поэтому в запросе SELECT они могут находиться только в списке выборки и во фразе ORDER BY.

Определение окна

Синтаксис оконных функций:

```
<function_name> (<function_arguments>)  
OVER(  
[PARTITION BY <expression_list>]  
[ORDER BY <order_by_list>]  
[{ROWS | RANGE} <frame_start> |  
{ROWS | RANGE} BETWEEN <frame_start> AND <frame_end>])
```

Конструкция в скобках после слова OVER называется определением окна. Последняя его часть, начинающаяся словом ROWS, называется определением фрейма.



Описание

Оконные функции обрабатывают наборы записей.

Оконные функции, в отличие от агрегатных, вычисляются для каждой строки.

Набор записей, обрабатываемых оконной функцией, строится следующим образом.

Вначале обрабатывается фраза `PARTITION BY`.

Отбираются все записи, для которых выражения, перечисленные в списке `expression_list`, имеют такие же значения, как в текущей записи. Это множество строк называется разделом (partition). Текущая строка также включается в раздел.

Фраза `PARTITION BY` логически и синтаксически идентична фразе `GROUP BY`, только ее аргументами являются выражения, а не имена выходных столбцов или числа.



Описание

При обработке каждой записи оконная функция просматривает все остальные записи, чтобы понять, какие из них попадают в один раздел с текущей.

Если фраза `PARTITION BY` отсутствует, то на данном шаге создается один раздел, содержащий все записи.

Далее раздел сортируется в соответствии с фразой `ORDER BY`, логически и синтаксически идентичной той же фразе в команде `SELECT`. Вновь ссылки на выходные столбцы по именам или по номерам не допускаются.

Если фраза `ORDER BY` опущена, то считается, что позиция каждой записи множества одинакова.



Оконные функции

Для каждой строки существует набор строк в её разделе, называемый **рамкой окна (оконным кадром, фреймом)**. У фрейма есть начало и конец.

По умолчанию, с указанием ORDER BY фрейм состоит из всех строк от начала раздела до текущей строки и строк, равных текущей по значению выражения ORDER BY.

Без ORDER BY фрейм по умолчанию состоит из всех строк раздела.

Пример:

```
SELECT salary, sum(salary) OVER () FROM empsalary;
```

salary	sum
5200	47100
5000	47100
3500	47100
4800	47100
3900	47100
4200	47100
4500	47100
4800	47100
6000	47100
5200	47100

(10 rows)

Так как в этом примере нет указания ORDER BY в предложении OVER, рамка окна содержит все строки раздела, а он, в свою очередь, без предложения PARTITION BY включает все строки таблицы; другими словами, сумма вычисляется по всей таблице и мы получаем один результат для каждой строки результата.



Оконные функции

Но если добавить ORDER BY, мы получим совсем другие результаты:

```
SELECT salary, sum(salary)
      OVER (ORDER BY salary)
FROM empsalary;
```

Здесь в сумме накапливаются зарплаты от первой (самой низкой) до текущей, включая повторяющиеся текущие значения (обратите внимание на результат в строках с одинаковой зарплатой).

salary	sum
3500	3500
3900	7400
4200	11600
4500	16100
4800	25700
4800	25700
5000	30700
5200	41100
5200	41100
6000	47100
(10 rows)	

Вопрос: что здесь является рамкой окна (фреймом) для каждой строки? Перечислите строки и укажите их фреймы, добавьте эти результаты в общие результаты



Оконные функции

Вот пример, показывающий, как сравнить зарплату каждого сотрудника со средней зарплатой его отдела:

```
SELECT depname, empno, salary, avg(salary) -- вывести номер отдела, номер работника, з/п работника,  
      OVER (PARTITION BY depname)          -- среднюю з/п по отделу: ... avg(salary) over (partition by...  
FROM empsalary;
```



Оконные функции

depname	empno	salary	avg
develop	11	5200	5020.0000000000000000
develop	7	4200	5020.0000000000000000
develop	9	4500	5020.0000000000000000
develop	8	6000	5020.0000000000000000
develop	10	5200	5020.0000000000000000
personnel	5	3500	3700.0000000000000000
personnel	2	3900	3700.0000000000000000
sales	3	4800	4866.6666666666666667
sales	1	5000	4866.6666666666666667
sales	4	4800	4866.6666666666666667

(10 rows)

Первые три столбца извлекаются непосредственно из таблицы empsalary, при этом для каждой строки таблицы есть строка результата.

В четвёртом столбце оказалось среднее значение, вычисленное по всем строкам, имеющим то же значение depname, что и текущая строка.

(Фактически среднее вычисляет та же функция avg, которую мы знаем как агрегатную, но предложение OVER превращает её в оконную, так что она обрабатывает лишь заданный набор строк.)

Задание: создайте таблицу, содержащую ФИО пациентов (8 пациентов), номера их палат (по 2-3 пациента в палате, температуру каждого пациента на каждый день в течение 3 дней). Напишите и выполните запрос (аналогичный представленному на предыдущем слайде) на основе оконных функций для сравнения температуры пациента со средней температурой по палате на каждый день. Добавьте эти результаты в общие результаты.



Оконные функции

Оконные функции разрешается использовать в запросе только в списке SELECT и предложении ORDER BY.

Во всех остальных предложениях, включая GROUP BY, HAVING и WHERE, они запрещены.

Это объясняется тем, что логически они выполняются после обычных агрегатных функций, а значит агрегатную функцию можно вызвать из оконной, но не наоборот.

Если нужно отфильтровать или сгруппировать строки после вычисления оконных функций, можно использовать вложенный запрос. Например:

```
SELECT depname, empno, salary, enroll_date
FROM
  (SELECT depname, empno, salary, enroll_date,
    rank() OVER (PARTITION BY depname ORDER BY salary DESC, empno) AS pos
  FROM empsalary
  ) AS ss
WHERE pos < 3;
```

Данный запрос покажет только те строки внутреннего запроса, у которых rank (порядковый номер) меньше 3.



Оконные функции

(Порядок ORDER BY для окна может даже не совпадать с порядком, в котором выводятся строки.) Например:

```
SELECT depname, empno, salary,  
       rank() OVER ( PARTITION BY depname  
                     ORDER BY salary DESC  
                     )  
FROM empsalary;
```

Функция rank выдаёт порядковый номер в разделе текущей строки для каждого уникального значения, по которому выполняет сортировку предложение ORDER BY.

У функции rank нет параметров, так как её поведение полностью определяется предложением OVER.

depname	empno	salary	rank
develop	8	6000	1
develop	10	5200	2
develop	11	5200	2
develop	9	4500	4
develop	7	4200	5
personnel	2	3900	1
personnel	5	3500	2
sales	1	5000	1
sales	4	4800	2
sales	3	4800	2

(10 rows)



Фрейм

Начало фрейма, которое в синтаксической диаграмме названо `frame_start`, может принимать следующие значения:

UNBOUNDED PRECEDING: самая первая запись раздела;

<N> PRECEDING: запись, предшествующая текущей в упорядоченном разделе и отстоящая от нее на N записей. Здесь <N> - целочисленное выражение, которое не может быть отрицательным и не может содержать агрегатные или другие оконные функции. 0 PRECEDING указывает на текущую запись;

CURRENT ROW: текущая строка;

<N> FOLLOWING: запись, следующая за текущей в упорядоченном разделе и отстоящая от нее на N записей.



ROWS/RANGE

Конец фрейма, `frame_end`, может принимать следующие значения:

`<N> PRECEDING;`

`CURRENT ROW;`

`<N> FOLLOWING;`

`UNBOUNDED FOLLOWING`: последняя запись раздела.

Начало должно предшествовать концу. Поэтому спецификация `ROWS BETWEEN CURRENT ROW AND 1 PRECEDING` некорректна.

Оконный фрейм можно определить в режиме `ROWS` или `RANGE`. Режим влияет на семантику `CURRENT ROW`.



ROWS/RANGE

В режиме ROWS CURRENT ROW указывает на саму текущую запись, а в режиме RANGE - на первую или последнюю запись с такой же позицией, что у текущей, в смысле сортировки, заданной фразой ORDER BY.

В режиме RANGE в определении фрейма можно использовать варианты UNBOUNDED ... или CURRENT ROW, но не PRECEDING.

Если часть frame_end опущена, предполагается CURRENT ROW.

Если опущено все определение фрейма, то предполагается определение RANGE UNBOUNDED PRECEDING.

Пример:

OVER (PARTITION BY a ORDER BY b ROWS BETWEEN UNBOUNDED PRECEDING AND 5 FOLLOWING)

Такое определение означает, что для каждой строки раздел образуют все записи с одинаковым значением в поле a. Затем раздел сортируется в порядке возрастания поля b, а фрейм содержит все записи от первой в разделе до пятой после текущей строки.



Оконные функции

Когда в запросе вычисляются несколько оконных функций для одинаково определённых окон, можно написать для каждой из них отдельное предложение OVER, но при этом оно будет дублироваться, что неизбежно будет провоцировать ошибки.

Поэтому лучше определение окна выделить в предложение WINDOW, а затем ссылаться на него в OVER. Например:

```
SELECT sum(salary) OVER w, avg(salary) OVER w
```

```
FROM empsalary
```

```
WINDOW w AS (PARTITION BY depname ORDER BY salary DESC);
```



Оконные функции

Пример.

Предположим, что руководство компании хочет усовершенствовать тарифную политику и с этой целью просит предоставить сведения о распределении количества проданных билетов на некоторые рейсы во времени.

Количество проданных билетов должно выводиться в виде накопленного показателя, суммирование должно производиться в пределах каждого календарного месяца.



Оконные функции

book_ref	book_date		month	day	count	
A60039	2016-08-22	12:02:00+08	8	22	1	кадр (frame)
554340	2016-08-23	23:04:00+08	8	23	2	
854C4C	2016-08-24	10:52:00+08	8	24	5	
854C4C	2016-08-24	10:52:00+08	8	24	5	
81D8AF	2016-08-25	10:22:00+08	8	25	6	
...						
8D6873	2016-08-31	17:09:00+08	8	31	59	раздел (partition)
E82829	2016-08-31	20:56:00+08	8	31	60	
ECA0D7	2016-09-01	00:48:00+08	9	1	1	
E3BD32	2016-09-01	04:44:00+08	9	1	2	
...						
EB11BB	2016-09-03	12:02:00+08	9	3	14	
19FE38	2016-09-03	17:42:00+08	9	3	16	
19FE38	2016-09-03	17:42:00+08	9	3	16	
536A3D	2016-09-03	19:19:00+08	9	3	18	
536A3D	2016-09-03	19:19:00+08	9	3	18	
02E6B6	2016-09-04	01:39:00+08	9	4	19	

(79 строк)

- Для примера был выбран рейс с идентификатором 1.
- Подсчет числа проданных билетов выполняется в пределах оконного кадра.
- Строки в выборке упорядочены по значениям столбца book_date.



Оконные функции

Запрос:

```
SELECT b.book_ref, b.book_date,  
       extract( 'month' from b.book_date ) AS month,  
       extract( 'day' from b.book_date ) AS day,  
       count( * ) OVER (  
         PARTITION BY date_trunc( 'month', b.book_date )  
         ORDER BY b.book_date  
       ) AS count  
FROM ticket_flights tf  
     JOIN tickets t ON tf.ticket_no = t.ticket_no  
     JOIN bookings b ON t.book_ref = b.book_ref  
WHERE tf.flight_id = 1  
ORDER BY b.book_date;
```

OVER - обязательное ключевое слово. Функция count становится оконной функцией.

Предложение PARTITION BY задает правило разбиения строк выборки на разделы.

Предложение ORDER BY предписывает порядок сортировки строк в разделах.



Оконные функции

В рассмотренном примере границы оконного кадра определялись по умолчанию.

Для указания границ оконного кадра предусмотрены различные способы.

Не только функция `count`, но и другие агрегатные функции (например, `sum`, `avg`) тоже могут применяться в качестве оконных функций.



Пример

С помощью оконной функции rank можно проранжировать аэропорты в пределах каждого часового пояса на основе их географической широты. Будем присваивать более высокий ранг тому аэропорту, который находится севернее.

```
SELECT airport_name, city, round( latitude::numeric, 2 ) AS ltd,  
       timezone, rank() OVER (  
       PARTITION BY timezone  
       ORDER BY latitude DESC ← сортировка в пределах каждого окна  
       )  
FROM airports  
WHERE timezone IN ( 'Asia/Irkutsk', 'Asia/Krasnoyarsk' )  
ORDER BY timezone, rank; ← сортировка в пределах всего запроса
```



Результат

	airport_name text	city text	ltd numeric	timezone text	rank bigint
1	Усть-Илимск	Усть-Илимск	58.14	Asia/Irkutsk	1
2	Усть-Кут	Усть-Кут	56.85	Asia/Irkutsk	2
3	Братск	Братск	56.37	Asia/Irkutsk	3
4	Иркутск	Иркутск	52.27	Asia/Irkutsk	4
5	Байкал	Улан-Удэ	51.81	Asia/Irkutsk	5
6	Норильск	Норильск	69.31	Asia/Krasnoyarsk	1
7	Стрежевой	Стрежевой	60.72	Asia/Krasnoyarsk	2
8	Богашёво	Томск	56.38	Asia/Krasnoyarsk	3
9	Емельяново	Красноярск	56.18	Asia/Krasnoyarsk	4
10	Абакан	Абакан	53.74	Asia/Krasnoyarsk	5
11	Барнаул	Барнаул	53.36	Asia/Krasnoyarsk	6
12	Горно-Алтайск	Горно-Алтайск	51.97	Asia/Krasnoyarsk	7
13	Кызыл	Кызыл	51.67	Asia/Krasnoyarsk	8



Предложение Window

Усложним запрос — для каждого аэропорта будем вычислять разницу между его географической широтой и широтой, на которой находится самый северный аэропорт в этом же часовом поясе.

Самый северный аэропорт в каждом часовом поясе, т. е. самая первая строка в каждом разделе, выбирается с помощью оконной функции `first_value`.

Эта функция получает доступ к первой строке оконного кадра, а не раздела.

Однако когда используются правила формирования оконного кадра по умолчанию, тогда его начало совпадает с началом раздела.



Предложение Window

```
SELECT airport_name, city, timezone, latitude,  
       first_value( latitude ) OVER tz AS first_in_timezone,  
       latitude - first_value( latitude ) OVER tz AS delta,  
       rank() OVER tz  
FROM airports  
WHERE timezone IN ( 'Asia/Irkutsk', 'Asia/Krasnoyarsk' )  
WINDOW tz AS ( PARTITION BY timezone ORDER BY latitude DESC )  
ORDER BY timezone, rank;
```

Если используется один и тот же способ формирования разделов и порядок сортировки строк в разделах - вводится предложение WINDOW.

Предложение WINDOW команды SELECT может находиться после фразы HAVING.



Предложение Window

```
SELECT count() OVER w, sum(b) OVER w, avg(b)  
      OVER (w ORDER BY c ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)  
FROM table1  
WINDOW w AS (PARTITION BY a)
```

Определенное таким образом окно можно использовать как есть. В примере выше функции count и sum так и делают. Но его можно и уточнить, как сделано в функции avg.

Синтаксически разница состоит в следующем: чтобы повторно использовать определение окна, имя этого окна следует указать после ключевого слова OVER без скобок.

Если же мы хотим расширить определение окна фразой ORDER BY или определением фрейма, то имя окна следует указать внутри скобок.

Если одно и то же определение окна используется несколько раз, то PostgreSQL оптимизирует выполнение запроса: строит разделы только один раз и использует результаты повторно.



Предложение Window

...

--[RECORD 4]-----+-----

airport_name	Иркутск
city	Иркутск
timezone	Asia/Irkutsk
latitude	52.268028
first_in_timezone	58.135
delta	-5.866972
rank	4

--[RECORD 5]-----+-----

airport_name	Байкал
city	Улан-Удэ
timezone	Asia/Irkutsk
latitude	51.807764
first_in_timezone	58.135
delta	-6.327236
rank	5

	airport_name text	city text	timezone text	latitude double precision	first_in_timezone double precision	delta double precision	rank bigint
1	Усть-Илимск	Усть-Илимск	Asia/Irkutsk	58.135	58.135	0	1
2	Усть-Кут	Усть-Кут	Asia/Irkutsk	56.85	58.135	-1.2849999999999966	2
3	Братск	Братск	Asia/Irkutsk	56.370556	58.135	-1.7644439999999975	3
4	Иркутск	Иркутск	Asia/Irkutsk	52.268028	58.135	-5.866971999999997	4
5	Байкал	Улан-Удэ	Asia/Irkutsk	51.807764	58.135	-6.327235999999999	5
6	Норильск	Норильск	Asia/Krasnoyarsk	69.311053	69.311053	0	1
7	Стрежевой	Стрежевой	Asia/Krasnoyarsk	60.716667	69.311053	-8.594386	2
8	Богашёво	Томск	Asia/Krasnoyarsk	56.380278	69.311053	-12.930775000000004	3
9	Емельяново	Красноярск	Asia/Krasnoyarsk	56.18	69.311053	-13.131053000000001	4
10	Абакан	Абакан	Asia/Krasnoyarsk	53.74	69.311053	-15.571053	5
11	Барнаул	Барнаул	Asia/Krasnoyarsk	53.363775	69.311053	-15.947278000000004	6
12	Горно-Алтайск	Горно-Алтайск	Asia/Krasnoyarsk	51.966667	69.311053	-17.344386	7
13	Кызыл	Кызыл	Asia/Krasnoyarsk	51.6694	69.311053	-17.641652999999998	8

Примеры

В базе данных для сайта торговли автомобилями есть таблица advertisement, содержащая информацию о рекламных объявлениях, созданных пользователями.

Пусть требуется проанализировать, сколько объявлений создано за период. Такой отчет генерируется следующим запросом:

```
WITH monthly_data AS (  
  SELECT date_trunc('month', advertisement_date) AS month, count(*) as cnt  
  FROM advertisement  
  GROUP BY date_trunc('month', advertisement_date)  
)  
SELECT to_char(month, 'YYYY-MM') as month, cnt,  
       sum(cnt) OVER (w ORDER BY month) AS cnt_year,  
       round(avg(cnt)  
             OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING), 1)  
AS mov_avg,  
       round(cnt / sum(cnt) OVER w * 100,1) AS ratio_year  
FROM monthly_data WINDOW w AS (PARTITION BY date_trunc('year', month));
```



Примеры

	количество	Накопленное количество	Скользящее среднее	Вклад текущего количества в общее количество
month	cnt	cnt_year	mov_avg	ratio_year
2014-01	42	42	40.3	5.8
2014-02	49	91	44.5	6.7
2014-03	30	121	56.8	4.1
2014-04	57	178	69.0	7.8
2014-05	106	284	73.0	14.6
2014-06	103	387	81.0	14.2
2014-07	69	456	86.0	9.5
2014-08	70	526	74.0	9.6
2014-09	82	608	60.6	11.3
2014-10	46	654	54.2	6.3
2014-11	36	690	49.8	5.0
2014-12	37	727	35.2	5.1
2015-01	48	48	32.5	84.2
2015-02	9	57	31.3	15.8

(14 rows)

Во фразе *WITH* данные агрегируются по месяцам. В главном запросе определяется окно *w*, предполагающее разбиение по годам. Это означает, что любая оконная функция, работающая с окном *w*, будет видеть записи за тот же год, что и текущая запись.

Первая оконная функция, *sum*, пользуется окном *w*. Поскольку присутствует фраза *ORDER BY*, у каждой записи есть определенная позиция в разделе. Определение фрейма опущено, поэтому предполагается фрейм *RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW*. Следовательно, функция вычисляет сумму значений по всем записям, начиная с начала года и до текущего месяца включительно, т. е. накопительный итог за год.

Вторая функция, *avg*, для каждой записи вычисляет скользящее среднее по пяти записям - две предшествующие, текущая и две последующие. Ранее определенное окно не используется, потому что при вычислении скользящего среднего год не принимается во внимание, важен только порядок записей.

Третья оконная функция, *sum*, пользуется тем же окном *w*. Вычисленная ею сумма значений за год является знаменателем в выражении, которое дает вклад текущего месяца в сумму.



Примеры

Существует ряд оконных функций, не являющихся агрегатными. Они служат для получения значений других записей в разделе, для вычисления ранга текущей записи относительно остальных и для генерации номеров строк.

Изменим предыдущий отчет, будем считать, что требуется вычислить две разности: между количеством объявлений в текущем месяце и предыдущем месяце того же года и в том же месяце прошлого года, а также ранг текущего месяца. Вот как выглядит такой запрос:

```
WITH monthly_data AS (  
  SELECT date_trunc('month', advertisement_date) AS month, count(*) as cnt  
  FROM advertisement  
  GROUP BY date_trunc('month', advertisement_date)  
)  
SELECT to_char(month, 'YYYY-MM') as month, cnt,  
       cnt - lag(cnt) OVER (ORDER BY month) as prev_m,  
       cnt - lag(cnt, 12) OVER (ORDER BY month) as prev_y,  
       rank() OVER (w ORDER BY cnt DESC) as rank  
FROM monthly_data  
WINDOW w AS (PARTITION BY date_trunc('year', month))  
ORDER BY month DESC;
```



Примеры

month	cnt	prev_m	prev_y	rank
2015-02	9	-39	-40	2
2015-01	48	11	6	1
2014-12	37	1		10
2014-11	36	-10		11
2014-10	46	-36		8
2014-09	82	12		3
2014-08	70	1		4
2014-07	69	-34		5
2014-06	103	-3		2
2014-05	106	49		1
2014-04	57	27		6
2014-03	30	-19		12
2014-02	49	7		7
2014-01	42			9

(14 rows)

Функция `lag` возвращает значение указанного выражения для записи, отстоящей на заданное число записей (по умолчанию 1) назад от текущей.

При первом вызове эта функция возвращает значение поля `cnt` из предыдущей записи, т. е. количество объявлений в предыдущем месяце.

Как видим, в феврале 2015 года опубликовано 9 объявлений - на 39 меньше, чем в январе 2015-го.

При втором вызове `lag` возвращает значение `cnt` для записи, отстоящей на 12 записей, т. е. на год назад от предыдущей. Число объявлений в феврале 2015 года на 42 меньше, чем в феврале 2014-го.

Функция `rank` возвращает ранг текущей строки внутри раздела. Имеется в виду ранг с промежутками, т. е. в случае, когда две записи занимают одинаковую позицию в порядке, заданном фразой `ORDER BY`, обе получают одинаковый ранг, а следующая получает ранг на две единицы больше.

То есть у нас будет две первые записи и одна третья.



Другие оконные функции

- `lead`: аналогична `lag`, но возвращает значение выражения, вычисленное для записи, отстоящей от текущей на указанное количество записей вперед;
- `first_value`, `last_value`, `nth_value`: возвращают значения выражения, вычисленные соответственно для первой, последней и `n`-й записи фрейма;
- `row_number`: возвращает номер текущей строки в разделе;
- `dense_rank`: возвращает ранг текущей строки без промежутков (плотный ранг);
- `percent_rank` и `cume_dist`: возвращает относительный ранг текущей строки. Разница между функциями состоит в том, что в первой числителем дроби является ранг, а во второй - номер строки;
- `ntile`: делит раздел на заданное количество равных частей и возвращает номер части, в которую попала текущая строка.



Оконные функции с группировкой и агрегированием

Поскольку оконные функции вычисляются после группировки, агрегатные функции могут быть аргументами оконных, но не наоборот.

Код `sum(count(*)) OVER()` правилен, как и код `sum(a) OVER(ORDER BY count(*))`.

Однако код `sum(count(*) OVER())` недопустим.

Например, чтобы вычислить ранги учетных записей продавцов по числу данных ими объявлений, можно выполнить такой запрос:

```
SELECT seller_account_id, dense_rank() OVER(ORDER BY count(*) DESC)
```

```
FROM advertisement
```

```
GROUP BY seller_account_id;
```

seller_account_id	dense_rank
26	1
128	2
28	2
126	2
...	



Продвинутые методы работы с SQL. Выборка первых записей

Часто бывает необходимо найти первые записи относительно какого-то критерия. Допустим, к примеру, что в базе данных car_portal нужно найти первое объявление для каждого идентификатора car_id в таблице advertisement.

В этом случае может помочь группировка.

Для реализации логики необходим подзапрос:

```
SELECT advertisement_id, advertisement_date, adv.car_id, seller_account_id
FROM advertisement adv
INNER JOIN
(SELECT car_id, min(advertisement_date) min_date
FROM advertisement
GROUP BY car_id) first
ON adv.car_id = first.car_id AND adv.advertisement_date = first.min_date;
```



Продвинутые методы работы с SQL. Выборка первых записей

Но если логика упорядочения настолько сложна, что для ее реализации функции min недостаточно, то такой подход работать не будет.

Проблему могут решить оконные функции, но они не всегда удобны:

```
SELECT DISTINCT first_value(advertisement_id) OVER w AS advertisement_id,  
min(advertisement_date) OVER w AS advertisement_date,  
car_id, first_value(seller_account_id) OVER w AS seller_account_id  
FROM advertisement  
WINDOW w AS (PARTITION BY car_id ORDER BY advertisement_date);
```

Здесь слово DISTINCT удаляет дубликаты, которые попали в одну группу.



Продвинутые методы работы с SQL. Выборка первых записей

PostgreSQL предлагает явный способ выбрать первую запись из каждой группы - ключевое слово `DISTINCT ON`. Синтаксически это выглядит так:

```
SELECT DISTINCT ON (<expression_list>) <Select-List>
```

...

```
ORDER BY <order_by_list>
```

Здесь для каждой уникальной комбинации значений из списка выражений `expression_list` команда `SELECT` возвращает только первую запись. Для определения того, что такое «первая», служит фраза `ORDER BY`.

Все выражения из списка `expression_list` должны присутствовать в списке `order_by_list`.



Продвинутые методы работы с SQL. Выборка первых записей

К рассматриваемой задаче эту идею можно применить следующим образом:

```
SELECT DISTINCT ON (car_id) advertisement_id, advertisement_date, car_id, seller_account_id  
FROM advertisement  
ORDER BY car_id, advertisement_date;
```

Этот код чище, проще для понимания и в большинстве случаев быстрее работает.



Продвинутые методы работы с SQL. Извлечение выборочных данных

Допустим, что база данных car_portal очень велика, а нам нужно собрать статистику по хранящимся в ней данным.

Абсолютной точности не требуется, будет достаточно и оценки. Но важно сделать это быстро. В PostgreSQL имеется способ опрашивать не всю таблицу, а случайную выборку из нее:

```
SELECT ... FROM <table> TABLESAMPLE <sampling_method> ( <argument> [, ...] )  
  
[ REPEATABLE ( <seed> ) ]
```



Продвинутые методы работы с SQL. Извлечение выборочных данных

Метод формирования выборки, `sampling_method`, может принимать значения `BERNOULLI` или `SYSTEM`. Оба эти метода входят в стандартный комплект поставки PostgreSQL, другие можно добавить в качестве расширений.

Оба метода принимают один аргумент - вероятность включения строки в выборку в процентах. Различие в том, что метод `BERNOULLI` предполагает полный просмотр таблицы, а алгоритм для каждой строки решает, включать ее в выборку или нет.

Метод `SYSTEM` делает то же самое, но на уровне блоков строк. Это значительно быстрее, но результаты не столь точны, потому что количество строк в разных блоках неодинаково.

Ключевое слово `REPEATABLE` задает начальное значение для случайных функций, используемых в алгоритме формирования выборки. Запросы с одинаковым начальным значением возвращают одинаковые результаты при условии, что данные в таблице не изменялись.



Продвинутые методы работы с SQL. Извлечение выборочных данных

Отбор строк происходит на этапе чтения данных из таблицы, еще до фильтрации, группировки и всех прочих операций.

Для иллюстрации увеличим количество записей в таблице advertisement, выполнив следующий запрос:

```
INSERT INTO advertisement (advertisement_date, car_id, seller_account_id)
SELECT advertisement_date, car_id, seller_account_id from advertisement;
INSERT 0 784
```

Здесь дублируются все записи в таблице - их число удваивается. Если выполнить этот запрос много раз, то таблица окажется достаточно большой. Это было выполнено 12 раз. После этого даже простой подсчет количества записей занимает заметное время:

```
\timing
```

```
Timing is on.
```

```
SELECT count(*) FROM advertisement;
count
```

```
-----
```

```
6422528
```

```
(1 row)
```

```
Time: 394.577 ms
```



Продвинутые методы работы с SQL. Извлечение выборочных данных

Теперь попробуем сделать то же самое для выборочных данных:

```
SELECT count(*) * 100 FROM advertisement TABLESAMPLE SYSTEM
```

```
(1);
```

```
?column?
```

```
-----
```

```
6248600
```

```
(1 row)
```

```
Time: 8.344 ms
```

Второй запрос гораздо быстрее, но результат не вполне точный. Мы умножили значение функции count на 100, потому что она возвращает процент строк.

При многократном выполнении запроса результат всякий раз будет разным, и иногда он будет сильно отличаться от истинного количества строк в таблице. Чтобы повысить качество, увеличим долю строк в выборке:

```
SELECT count(*) * 10 from advertisement TABLESAMPLE SYSTEM
```

```
(10);
```

```
?column?
```

```
-----
```

```
6427580
```

```
(1 row)
```

```
Time: 92.793 ms
```

Время работы увеличилось пропорционально.



Продвинутые методы работы с SQL. Функции, возвращающие множества

Функции в PostgreSQL могут возвращать не только одиночные значения, но и отношения. Они называются функциями, возвращающими множество.

Вот типичная задача, встающая перед всяким разработчиком на SQL: сгенерировать последовательность целых чисел, по одному в записи. У такой последовательности, или отношения, много применений. Предположим, к примеру, что в базе `car_portal` нужно подсчитать количество автомобилей, выпущенных в каждом году от 2010-го до 2015-го, показав нули, если автомобилей за какой-то год в базе нет. Простой `SELECT` из одной лишь таблицы `car` задачу не решит.

Невозможно показать данные, которых нет в таблице. Поэтому было бы полезно иметь таблицу, содержащую числа от 2010 до 2015. Тогда мы смогли бы выполнить внешнее соединение между ней и таблицей результатов.

Можно было бы подготовить такую таблицу заранее, но это не очень хорошо, потому что сколько понадобится записей, заранее неизвестно, а создавать слишком большую таблицу - значит впустую транжирить место на диске.



Продвинутые методы работы с SQL. Функции, возвращающие множества

Но существует функция `generate_series`, специально предназначенная для этой цели. Она возвращает множество целых чисел между двумя заданными.

Для решения задачи надо было бы написать такой запрос:

```
SELECT years.manufacture_year, count(car_id)
FROM generate_series(2010, 2015) as years (manufacture_year)
LEFT JOIN car ON car.manufacture_year = years.manufacture_year
GROUP BY years.manufacture_year
ORDER BY 1;
```

manufacture_year	count
2010	11
2011	12
2012	12
2013	21
2014	16
2015	0

(6 rows)

Здесь функция `generate_series` возвращает шесть целых чисел от 2010 до 2015. Этому множеству сопоставлен псевдоним `years`. Таблица `car` соединяется слева с этим множеством, и теперь мы видим в результирующем наборе все годы.

Выполните запрос, используя `generate_series(число1, число2)`, где в качестве чисел «число1», «число2» задаются начальный и конечный номера дней в месяце (например, 2, 10 - со 2 по 10 дни), который выводит соответствующие сгенерированным дням количества улетевших пассажиров. Запрос и его результаты поместите в общий отчет.



Продвинутые методы работы с SQL. Функции, возвращающие множества

При вызове `generate_series` можно задать шаг:

```
SELECT * FROM generate_series(5, 11, 3);
```

`generate_series`

5
8
11

(3 rows)

шаг

Функция `generate_series` может также возвращать множество значений типа `timestamp`:

```
SELECT * FROM generate_series('2015-01-01'::date, '2015-01-31'::date, interval '7 days');
```

`generate_series`

2015-01-01 00:00:00-05
2015-01-08 00:00:00-05
2015-01-15 00:00:00-05
2015-01-22 00:00:00-05
2015-01-29 00:00:00-05

(5 rows)

шаг

Есть еще две функции, возвращающие множества, предназначенные для работы с массивами:

➤ **`generate_subscripts`**: генерирует набор индексов для указанной размерности заданного массива. Полезна для перечисления элементов массива в команде SQL;

➤ **`unnest`**: преобразует заданный массив в набор строк, каждая запись которого соответствует элементу массива.



Продвинутые методы работы с SQL. Функции, возвращающие множества

Функции, возвращающие множества, называют еще табличными функциями.

Табличная функция может возвращать множество строк predetermined типа, например `generate_series` возвращает значение типа `setof int` или `setof bigint` (в зависимости от типа входного аргумента).

А может возвращать множество записей абстрактного типа. Это позволяет возвращать таблицы с разным числом столбцов, зависящим от аргументов.

SQL требует, чтобы структура строк всех входных отношений была определена таким образом, чтобы структура результирующего множества также была определена.

Именно поэтому использование табличных функций в запросе может сопровождаться определением структуры строки возвращаемого множества:

```
function_name () [AS] alias (column_name column_type [, ...])
```



Продвинутые методы работы с SQL. Функции, возвращающие множества

Результаты нескольких табличных функций можно объединять, как если бы они были соединены по позиции строки. Для этого служит конструкция ROWS FROM:

`ROWS FROM (function_call [...]) [[AS] alias (column_name [...])]`

Эта конструкция возвращает отношение, поэтому ее можно использовать во фразе FROM, как любое другое отношение. Количество строк равно размеру самого большого результата перечисленных функций. Столбцы соответствуют функциям во фразе ROWS FROM. Если какая-то функция возвращает меньше строк, чем другие, то отсутствующие значения будут равны NULL:

```
SELECT foo.a, foo.b FROM  
ROWS FROM (generate_series(1,3), generate_series(1,7,2)) AS foo(a, b);
```

```
a | b  
---+---  
1 | 1  
2 | 3  
3 | 5  
  | 7  
(4 rows)
```



Продвинутые методы работы с SQL. Латеральные подзапросы

Удобно использовать подзапросы в списке выборки. Например, в командах SELECT с их помощью можно создавать вычисляемые атрибуты при опросе таблицы.

Снова обратимся к базе данных car_portal. Пусть требуется для каждого автомобиля в таблице car оценить его возраст, сравнив с возрастом других автомобилей той же модели. И кроме того, мы хотим получить общее количество автомобилей той же модели. Оба дополнительных поля можно сгенерировать скалярными подзапросами:

```
SELECT car_id, manufacture_year,  
CASE WHEN manufacture_year <= (  
  SELECT avg(manufacture_year) FROM car_portal_app.car WHERE car_model_id  
= c.car_model_id)  
  THEN 'old' ELSE 'new' END as age,  
(SELECT count(*) FROM car WHERE car_model_id = c.car_model_id)  
AS same_model_count  
FROM car c;
```

Легко видеть, что запрос мог бы работать лучше, если бы мы включили в список выборки подзапроса два столбца, но, увы, это невозможно. Скалярные запросы могут возвращать только один столбец.



Продвинутые методы работы с SQL. Латеральные подзапросы

car_id	manufacture_year	age	same_model_count
--------	------------------	-----	------------------

1	2008	old	3
---	------	-----	---

2	2014	new	6
---	------	-----	---

3	2014	new	2
---	------	-----	---

...

(229 rows)

Эти подзапросы интересны тем, что могут ссылаться на главную таблицу в своей фразе WHERE.

С помощью подобных подзапросов легко добавить в запрос дополнительные столбцы. Но есть и проблема - производительность.

Таблица car просматривается сервером один раз в главном запросе, а затем еще два раза для каждой выбранной строки, т. е. для столбцов age и same_model_count.



Продвинутые методы работы с SQL. Латеральные подзапросы

Можно независимо вычислить эти агрегаты по одному разу для каждой модели, а затем соединить результаты с таблицей car:

```
SELECT car_id, manufacture_year,  
CASE WHEN manufacture_year <= avg_year THEN 'old' ELSE 'new' END as age,  
same_model_count FROM car  
INNER JOIN (  
SELECT car_model_id, avg(manufacture_year) avg_year, count(*) same_model_count  
FROM car  
GROUP BY car_model_id) subq  
USING (car_model_id);  
car_id | manufacture_year | age | same_model_count
```

```
1      | 2008              | old | 3
```

```
2      | 2014              | new | 6
```

```
3      | 2014              | new | 2
```

```
...
```

```
(229 rows)
```

Результат тот же самый, а запрос выполняется в 20 раз быстрее. Однако этот запрос хорош только тогда, когда нужно выбрать из базы данных много строк. Если нужно получить информацию только об одном автомобиле, то первый запрос будет быстрее.



Продвинутые методы работы с SQL. Латеральные подзапросы

Существует еще один способ использования подзапросов. Он сочетает в себе преимущества подзапросов в списке выборки, способных обращаться к главной таблице из фразы WHERE, с подзапросами во фразе FROM, которые могут возвращать несколько столбцов.

Если поместить ключевое слово LATERAL перед подзапросом во фразе FROM, то он сможет ссылаться на любой элемент, предшествующий ему во фразе FROM.

Запрос выглядит следующим образом:

```
SELECT car_id, manufacture_year,  
CASE WHEN manufacture_year <= avg_year THEN 'old' ELSE 'new' END as age,  
same_model_count  
FROM car c,  
LATERAL (  
SELECT avg(manufacture_year) avg_year, count(*) same_model_count  
FROM car  
WHERE car_model_id = c.car_model_id) subq;
```



Продвинутые методы работы с SQL. Латеральные подзапросы

car_id	manufacture_year	age	same_model_count
1	2008	old	3
2	2014	new	6
3	2014	new	2

...

(229 rows)

Этот запрос работает приблизительно в два раза быстрее первого, он оптимален, когда нужно выбрать из таблицы car всего одну строку.

Синтаксис JOIN также допускается в латеральных подзапросах, хотя в большинстве случаев условие соединения каким-то образом включается во фразу WHERE подзапросов.

Употреблять LATERAL с функциями, возвращающими множества, нет необходимости. Все функции, упомянутые во фразе FROM, и так могут использовать результаты любых предшествующих функций или подзапросов:

SELECT a, b

FROM generate_series(1,3) AS a, generate_series(a, a+2) AS b;

a	b
1	1
1	2
1	3
2	2
2	3
2	4
3	3
3	4
3	5

(9 rows)

Здесь первая функция, с псевдонимом a, возвращает три строки. Для каждой из них вызывается вторая функция, возвращающая еще три строки.



Продвинутые методы работы с SQL. Дополнительные средства группировки

Базы данных часто служат источником данных для различных отчетов. А в отчетах принято в одной и той же таблице показывать подытоги, итоги и общие итоги, подразумевающие группировку и агрегирование.

Рассмотрим отчет о количестве объявлений в разрезе марок и кварталов, в котором требуется также показать итоги по каждому кварталу (суммарно по всем маркам) и общий итог.

Вот как отбираются данные для такого отчета:

```
SELECT to_char(advertisement_date, 'YYYY-Q') as quarter, make, count(*)  
FROM advertisement a  
  INNER JOIN car c ON a.car_id = c.car_id  
  INNER JOIN car_model m ON m.car_model_id = c.car_model_id  
GROUP BY quarter, make;
```

quarter	make	count
2014-4	Peugeot	12
2014-2	Daewoo	8
2014-4	Skoda	5
...		



Продвинутые методы работы с SQL. Дополнительные средства группировки

Чтобы вычислить итоги, понадобятся дополнительные запросы:

```
SELECT to_char(advertisement_date, 'YYYY-Q') as quarter, count(*)  
FROM advertisement a  
  INNER JOIN car c ON a.car_id = c.car_id  
  INNER JOIN car_model m ON m.car_model_id = c.car_model_id  
GROUP BY quarter;
```

```
SELECT count(*)  
FROM advertisement a  
  INNER JOIN car c ON a.car_id = c.car_id  
  INNER JOIN car_model m ON m.car_model_id = c.car_model_id;
```

Соединения в этих запросах излишни - они не влияют на результат, но они здесь для того, чтобы было видно, что запросы фактически такие же, как первый, отличается только группировка.



Продвинутые методы работы с SQL. Дополнительные средства группировки

PostgreSQL позволяет объединить все три запроса в один с помощью специальной конструкции **GROUP BY GROUPING SETS**:

```
SELECT to_char(advertisement_date, 'YYYY-Q')
as quarter, make, count(*)
FROM advertisement a
INNER JOIN car c ON a.car_id = c.car_id
INNER JOIN car_model m
ON m.car_model_id = c.car_model_id
GROUP BY GROUPING SETS ((quarter, make),
(quarter), ())
ORDER BY quarter NULLS LAST, make NULLS
LAST;
```

quarter	make	count
2014-1	Alfa Romeo	2
2014-1	Audi	5
...		
2014-1	Volvo	9
2014-1		121 <- Это кварталный подытог
2014-1		
2014-2	Audi	18
2014-2	Citroen	40
...		
2014-2	Volvo	12
2014-2		266 <- Это кварталный подытог
2014-2		
2014-3	Audi	11
...		
2015-1	Volvo	4
2015-1		57 <- Это кварталный подытог
2015-1		
		784 <- Это общий итог

Продвинутые методы работы с SQL. Дополнительные средства группировки

Благодаря конструкции GROUPING SETS ((quarter, make), (quarter), ()) запрос работает как UNION ALL одинаковых запросов с разными фразами GROUP BY:

- GROUP BY quarter, make;
- GROUP BY quarter - здесь поле make будет иметь значение NULL;
- все строки помещаются в одну группу, оба поля - quarter и make - будут иметь значение NULL.

Фраза GROUP BY принимает не только выражения, но и группирующие элементы, которые могут быть выражениями или конструкциями типа GROUPING SETS.



Продвинутые методы работы с SQL. Дополнительные средства группировки

Два других группирующих элемента - ROLLUP и CUBE.

- ROLLUP (a, b, c) эквивалентно GROUPING SETS ((a, b, c), (a, b), (c), ()).
- CUBE (a, b, c) эквивалентно GROUPING SETS ((a, b, c), (a, b), (a, c), (b, c), (a), (b), (c), ()) - все возможные комбинации аргументов.

Разрешено также использовать CUBE и ROLLUP внутри GROUPING SETS, а равно комбинировать различные группирующие элементы в одной фразе GROUP BY, например:

GROUP BY a, CUBE (b, c). Это эквивалентно GROUP BY GROUPING SETS ((a, b, c), (a, b), (a, c), (a)) - по сути дела, комбинация группирующих элементов.

Хотя GROUPING SETS логически эквивалентно UNION ALL результатов запросов с разными фразами GROUP BY, реализована эта конструкция не так, как теоретико-множественная операция.

PostgreSQL просматривает таблицы и выполняет соединения и фильтрацию только один раз. Следовательно, применение таких методов группировки может повысить производительность запросов.



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Есть несколько агрегатных функций, которые выполняются специальным образом.

Первая группа таких функций называется агрегатами по упорядоченному множеству (ordered set aggregates). Они учитывают не только значения выражений, переданных в качестве аргументов, но и их порядок. Такие агрегаты имеют отношение к статистике и вычислению процентилей.

Процентиль - это значение группы, в котором процентное значение общих значений равно этому значению или меньше его. Например, если некоторое значение равно 95-му процентилю, значит, оно больше 95 процентов остальных значений.

PostgreSQL позволяет вычислить непрерывный или дискретный процентиль с помощью функций `percentile_cont` и `percentile_disc` соответственно.

Дискретный процентиль совпадает с одним из значений группы, а непрерывный - результат интерполяции двух существующих значений. Есть возможность вычислить один процентиль для заданной процентной доли или сразу несколько процентов для заданного массива долей.



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Пример: запрос о распределении количества объявлений по автомобилям:

```
SELECT percentile_disc(ARRAY[0.25, 0.5, 0.75]) WITHIN GROUP  
(ORDER BY cnt)  
FROM (SELECT count(*) cnt FROM advertisement GROUP BY car_id) subq;  
percentile_disc
```

{2,3,5}

(1 row)

Результат означает, что для 25 процентов автомобилей существует не более двух объявлений, для 50 процентов - три объявления и для 75 процентов - пять объявлений



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Синтаксис агрегатных функций по упорядоченному множеству отличается от обычных агрегатных функций, в нем используется специальная конструкция WITHIN GROUP (ORDER BY expression).

Здесь выражение expression - фактически аргумент функции. На результат функции влияет не только порядок строк, но и значения этих выражений. В противоположность фразе ORDER BY в команде SELECT допускается только одно выражение, и ссылок на номера результирующих столбцов быть не должно.

Еще одна агрегатная функция по упорядоченному множеству называется mode. Она возвращает самое часто встречающееся значение в группе. Если два значения встречаются одинаковое число раз, то возвращается первое из них.

Например, следующий запрос возвращает идентификатор самой частой встречающейся модели автомобиля в базе данных:

```
SELECT mode() WITHIN GROUP (ORDER BY car_model_id)
FROM car;
```

```
mode
-----
64
(1 row)
```

Напишите запрос, который выводит наиболее часто встречающуюся в БД «Авиарейсы» модель самолета, используя mode. Результат поместите в общий результат работы.



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Чтобы получить тот же результат без этой функции, понадобилось бы самосоединение или сортировка и ограничение количества результатов. То и другое обошлось бы дороже.

Другая группа агрегатов с таким же синтаксисом - агрегатные функции по гипотетическому множеству: `rank`, `dense_rank`, `percent_rank` и `cume_dist`. Существуют оконные функции с такими же именами.

Но оконные функции не принимают аргумента и возвращают результат для текущей строки. Для агрегатных функций понятия текущей строки не существует, потому что они вычисляются для группы. Зато они принимают аргумент: значение гипотетической текущей строки.

Например, агрегатная функция `rank` возвращает ранг заданного значения в упорядоченном множестве, если бы оно в этом множестве присутствовало:

```
SELECT rank(2) WITHIN GROUP (ORDER BY a)
FROM generate_series(1,10,3) a;
rank
-----
2
(1 row)
```

В данном случае значения 2 нет в результате функции `generate_series` (она возвращает множество 1..4..7..10). Но если бы оно там присутствовало, то занимало бы вторую позицию.



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Еще один аспект агрегатных функций - фраза FILTER. Она по заданному условию отфильтровывает строки, передаваемые агрегатной функции.

Пусть, например, требуется подсчитать количество автомобилей для каждой модели и каждого значения количества дверей. Если бы мы сгруппировали данные по этим двум полям, то результат получился бы правильным, но не очень удобным для отчетности:

```
SELECT car_model_id, number_of_doors, count(*)
```

```
FROM car
```

```
GROUP BY car_model_id, number_of_doors;
```

car_model_id	number_of_doors	count
47	4	1
42	3	2
76	5	1
52	5	2
...		



Продвинутые методы работы с SQL. Дополнительные виды агрегирования

Благодаря фразе FILTER результат становится гораздо понятнее:

```
SELECT car_model_id,  
count(*) FILTER (WHERE number_of_doors = 2) doors2,  
count(*) FILTER (WHERE number_of_doors = 3) doors3,  
count(*) FILTER (WHERE number_of_doors = 4) doors4,  
count(*) FILTER (WHERE number_of_doors = 5) doors5  
FROM car_portal_app.car GROUP BY car_model_id;
```

car_model_id	doors2	doors3	doors4	doors5
43	0	0	0	2
8	0	0	1	0
11	0	2	1	0
80	0	1	0	0

... Этот запрос не считает автомобили с количеством дверей меньше двух и больше пяти. Того же результата можно достичь и таким образом:

```
count(CASE WHEN number_of_doors = 2 THEN 1 END) doors2...
```

Но вариант с фразой FILTER проще.



Упражнения (блок 1 – оконные функции). Номер варианта = (Номер_по_журналу % 10) + 1, % - остаток от деления

Столбец: задача, строка: номер варианта	1. С помощью запроса, использующего концепцию оконных функций, выведите накопленные суммы продаж билетов по q в каждом месяце.	2. С помощью запроса, использующего концепцию оконных функций, по каждой модели самолета, выполняющего рейсы, выведите ежедневное количество рейсов, накопленное количество рейсов, скользящее среднее количества рейсов для фрейма p (учитываем, что фрейм находится внутри раздела), относительное количество рейсов, приходящееся на весь рассматриваемый период полетов.	3. С помощью запроса, использующего концепцию оконных функций, вычислите ранги моделей самолетов по
1	q = «парам дней»	p - по умолчанию	числу совершенных ими полетов
2	q = «неделям»	p - предыдущая, текущая и следующая строки	средней длительности полетов
3	q = «часам» (за каждый час)	p - от начала раздела до второй строки под текущей	количеству мест класса «Эконом»
4	q = «неделям»	p - от текущей строки до 3 следующих строк	количеству мест класса «Бизнес»
5	q = «парам дней»	p -от текущей строки до конца раздела	количеству мест класса «Комфорт»
6	q = «часам» (за каждый час)	p - от второй сверху от текущей до третьей после текущей строки	средней цене билета
7	q = «парам дней»	p - от начала раздела до третьей строки под текущей	средней стоимости перелетов
8	q = «неделям»	p - предыдущая, текущая и 2 следующих строки	количеству пассажиров в неделю
9	q = «часам» (за каждый час)	p - по умолчанию	средней задержке отправления
10	q = «неделям»	p - текущая строка и 4 следующие	количеству пассажиров в месяц



Упражнения (блок 2 – продвинутые методы). Номер варианта = (Номер_по_журналу % 10) + 1, % - остаток от деления

Столбец: задача, строка: номер варианта	1. Увеличьте количество записей в таблице t в p раз, аналогично увеличению количества записей в таблице advertisement. Выполните подсчет строк в таблице t, используя выборку из нее. Вероятность включения строки в выборку в процентах = q.	2. Придумайте и реализуйте запрос с латеральным подзапросом для ранее созданной таблицы комментариев чата (хранящей дерево комментариев), в соответствии со своим вариантом.	3. а) Напишите запрос, который для каждой модели самолета выводит количество мест q. Используйте вариант с FILTER и вариант без FILTER (просто CASE WHEN). ИЛИ б) Напишите запрос, который находит p номер билета для каждого бронирования (используйте DISTINCT ON, а также альтернативные варианты).	4. Напишите запрос, который возвращает ранг числа x в множестве, которое начинается числом a, ограничено числом b, шаг между числами - h. Выведите все числа этого множества.	5. Напишите запрос о распределении количества билетов по рейсам для p и q процентов.
1	t = flights, p=4, q=1	===//===	а) q = «бизнес-класса и эконом-класса»	x = 56, a = 2, b = 100, h = 1	p = 10, q = 90
2	t = tickets, p=8, q=2	===//===	а) q = «эконом-класса и комфорт-класса»	x = 40, a = 4, b = 100, h = 2	p = 25, q = 80
3	t = seats, p=16, q=10	===//===	а) q = «бизнес-класса, эконом-класса и комфорт-класса»	x = 35, a = 10, b = 1000, h = 5	p = 50, q = 60
4	t = aircrafts (_data), p=32, q=3	===//===	а) q = «бизнес-класса и комфорт-класса»	x = 27, a = 1, b = 90, h = 5	p = 95, q = 1
5	t= boarding_passes, p = 16, q = 5	===//===	б) p = «минимальный»	x = 12, a = 1, b = 120, h = 4	p = 34, q = 15
6	t = ticket_flights, p=8, q=4	===//===	б) p = «максимальный»	x = 18, a = 1, b = 80, h = 2	p = 3, q = 97
7	t = flights, p=8, q=10	===//===	б) p = “первый с единицей в конце”	x = 21, a = 1, b = 100, h = 6	p = 10, q = 89
8	t = tickets, p=16, q=7	===//===	б) p = “первый с единицей в начале”	x = 76, a = 50, b = 5000, h = 10	p = 14, q = 78
9	t = seats, p=16, q=12	===//===	а) q = «бизнес-класса и эконом-класса»	x = 37, a = 12, b = 99, h = 4	p = 12, q = 81
10	t = ticket_flights, p=4, q=1	===//===	б) p = «минимальный»	x = 15, a = 1, b = 50, h = 2	p = 3, q = 99

6. Общее задание:
Используя ROLLUP, GROUPING SETS, CUBE (напишите одни и те же запросы разными способами) вычислите суммы продаж билетов на авиарейсы по моделям самолетов, дням, месяцам и сумму продаж за весь период.

